

KineticPulse

A Multimodal Edge-AI Prototype for Fall Detection and Caregiver Escalation

Youngwon Cho[✉], Chang-Ting Zhong[✉], Hao-Yuan Weng[✉], Yule Xu[✉], Ren-Yi Huang[✉],
Yuanhao Chen[✉], Yiyuan Chen[✉], and Yong Zhe Sam[✉]

School of Computation, Information and Technology, Technical University of Munich

✉ youngwon.cho@tum.de

✉ tim.zhong@tum.de

✉ hao-yuan.weng@tum.de

✉ yule.xu@tum.de

✉ go57bin@mytum.de

✉ ie.yuanhao@tum.de

✉ yiyuan.chen@tum.de

✉ go36yev@mytum.de

Final Project Report

Abstract — KineticPulse is our prototype for detecting a possible fall and helping a caregiver respond with useful context. The system combines camera-based inference on an NVIDIA Jetson Orin Nano with an ESP32-S3 wristband for photoplethysmography (PPG) and motion sensing. The repository contains a four-class YOLOv8 detector, pose estimation, a two-stream spatial-temporal graph model, rule-based multimodal fusion, spoken verification, webhooks, and WebRTC caregiver clients. At revision 07631fe, the wristband firmware sends contract-conformant telemetry over TCP, the caregiver dashboard consumes the Jetson's live monitoring endpoint, and the reference detector checkpoint reaches a validation mAP50 of 0.885. These results show that the individual paths work, but they do not yet prove real-world fall-detection performance: the detector split contains no held-out sitting instances, PPG is still being brought up on real hardware, and no end-to-end latency or endurance measurements have been archived. We therefore present KineticPulse as an integrated laboratory prototype, rather than as a finished medical or emergency system.

Contents

Project at a Glance	3
1 Introduction	4
1.1 Project goals	4
2 Background and Related Work	5
3 System Design	6
3.1 Architecture	6
3.2 Visual pipeline	6
3.3 Wearable path	6
3.4 Fusion and escalation	7
3.5 Voice, WebRTC, and notification	8
3.6 Web monitoring dashboard	8
4 Implementation	10
5 Verification and Current Results	12
5.1 Evaluation plan	13
6 Limitations and Future Work	14
7 Conclusion	15
References	16

Project at a Glance

KineticPulse is best understood as an integration prototype. Its software already connects perception, decision-making, user verification, and caregiver access. The remaining challenge is to replace simulated inputs with the real wristband and then measure the complete chain under controlled conditions.

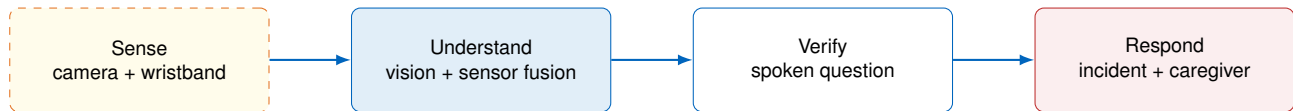


Figure 1 The intended user journey. The physical wristband segment is not yet validated end to end.

Table 1 Current evidence boundary at repository revision 07631fe.

Area	What is available now	What is still needed
Perception	Detector, pose estimation, temporal model, tracking, and packaged checkpoints.	A frozen four-class test set, reproducible metrics, difficult negatives, and target-camera trials.
Wearable	TCP receiver, typed sensor state, synthetic PPG tests, and an ESP32 connectivity scaffold.	Real MAX30102 and IMU acquisition, one compatible packet schema, timestamps, and signal-quality checks.
Decision	Inspectable fusion rules, sensor-availability handling, and three escalation tiers.	Threshold calibration with synchronized real inputs plus false-alarm and missed-event measurements.
Interaction	Voice interfaces, incident sessions, web and mobile clients, WebRTC, and webhook control flow.	Real-room speech trials, authenticated cross-network video, and confirmed notification delivery.
Verification	84 automated tests pass and one dataset-dependent detector smoke test is skipped.	End-to-end hardware, latency, reliability, battery, recovery, privacy, and field evidence.

Reading rule. A component described as implemented exists in software. It should be treated as demonstrated only when the report also gives a reproducible test setup and measured result. This distinction is used consistently throughout the report.

The report first motivates the design, then follows the path from sensing to escalation. It closes with the evidence we can currently support and the shortest route to a credible end-to-end evaluation.

1 Introduction

Falls are a serious problem, especially for older adults. The World Health Organization estimates that they cause about 684,000 deaths and 37.3 million injuries requiring medical attention each year [1]. Detecting a fall is only one part of the challenge. A useful system must also decide whether the person needs help, remain understandable when sensors disagree, and send enough information to a caregiver without exposing more private data than necessary.

KineticPulse grew from a simple question: can a camera and a small wearable support each other when either one is uncertain? A camera can see posture and movement, but it is limited by lighting, occlusion, and field of view. A wristband can travel with the user and capture body signals, but it depends on contact, charging, and wireless communication. Our design brings both streams to an NVIDIA Jetson, makes a transparent rule-based decision, and asks the user for spoken confirmation when there is time to do so.

This report describes what we built, what our automated tests establish, and what still needs to be demonstrated. We deliberately avoid presenting planned hardware or unmeasured performance as a finished result. KineticPulse is not a medical device, does not diagnose seizures or cardiac conditions, and is not connected to a public emergency number. It is a student prototype for exploring multimodal fall detection and caregiver escalation.

1.1 Project goals

Our goals were to:

- process camera data locally on an edge computer;
- combine visual evidence with inertial and PPG measurements from a wearable;
- keep the escalation logic inspectable instead of hiding it in a second black-box model;
- ask the observed person whether help is needed for ambiguous events; and
- expose an incident session to web and mobile caregiver clients.

The software repository covers all five goals at different maturity levels. Visual inference, temporal features, fusion, voice interfaces, webhooks, WebRTC, and the caregiver dashboard are implemented, and the wristband now speaks the documented TCP contract. The remaining gaps are in sensing rather than in plumbing: the optical heart-rate sensor is still being brought up on real hardware, and the assembled wearable is a clip-on breadboard prototype rather than a wrist-worn device.

2 Background and Related Work

Fall-detection approaches are usually grouped into wearable, ambient, and vision-based systems [2, 3]. Wearable systems often use accelerometers and gyroscopes. They work outside a fixed camera view, but they can be forgotten, worn incorrectly, or discharged. Reviews of wearable systems also point to the persistent difficulty of distinguishing actual falls from everyday movements that look similar in a short sensor window [4].

Vision-based systems use RGB, depth, or thermal cameras to recognize posture and motion. Deep-learning methods can capture whole-body context and do not require the user to remember a device. Their weak points are equally practical: occlusion, poor lighting, changing viewpoints, and the privacy implications of cameras in living spaces [5]. A person lying on the floor may have fallen, may be exercising, or may simply be outside the situation represented by the training data.

Multimodal systems try to combine complementary evidence. A visible fall together with an inertial impact or unusual physiological reading is more informative than either observation alone. Recent research and datasets show increasing interest in this direction [6, 7]. Those published results provide motivation, but they cannot be reused as KineticPulse results because the sensors, participants, environments, and models are different.

For the visual path, KineticPulse uses YOLOv8 interfaces for object detection and pose estimation [8]. The detector distinguishes *fallen*, *falling*, *stand*, and *sitting*. Pose sequences are passed to a two-stream spatial-temporal graph-convolutional network (ST-GCN), an approach that represents body joints as a graph over space and time [9]. The wearable is planned around an ESP32-S3 [10], a MAX30102 optical sensor [11], and an IMU. PPG is promising for wearables but sensitive to motion, contact pressure, ambient light, and processing choices [12]; our lightweight processor should therefore be treated as a prototype rather than a clinical measurement method.

The Jetson keeps inference close to the sensors, following the general edge-computing idea of reducing dependence on distant services for interactive tasks [13, 14]. Spoken verification uses a *faster-whisper* integration [15], based on the Whisper model family [16]. Caregiver video is delivered through WebRTC [17]; ICE is used to establish a path across network boundaries [18]. These technologies provide useful building blocks, but the surrounding application still has to implement authentication, recovery, and privacy controls.

3 System Design

3.1 Architecture

The system has three physical parts: a wearable, an edge node, and caregiver clients. The wearable transmits three-axis motion and heart-rate events to the Jetson over Wi-Fi as newline-delimited JSON on TCP port 5555. A camera connects directly to the Jetson. The Jetson performs visual inference, receives wearable packets, combines the evidence, and creates an incident when an escalation rule is satisfied. It also publishes continuous vitals as JSON on port 8790 and hosts WebRTC signaling on port 8787, so web and mobile clients can show a live status view and join a video session; a generic webhook can additionally notify a configured service. In the laboratory the clients reach the Jetson over a Tailscale mesh rather than a public address.

Figure 2 shows this structure. Solid blue blocks exist in the repository and have been exercised. Dashed orange blocks or links still require hardware bring-up or deployment validation.

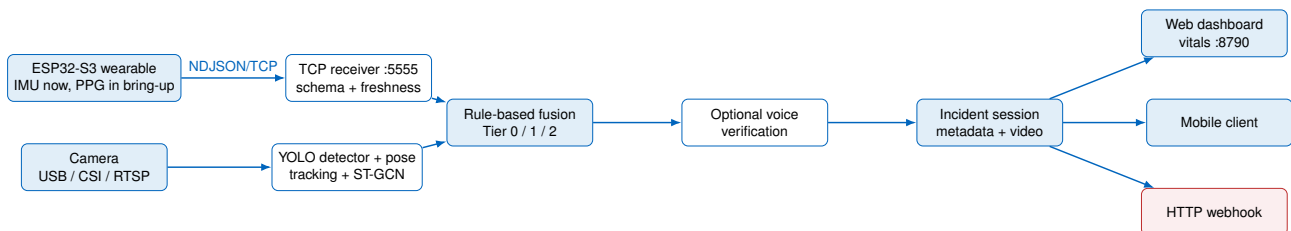


Figure 2 KineticPulse architecture. All links shown are implemented; the wearable carries real motion data, while its optical heart-rate path is still in bring-up.

3.2 Visual pipeline

Each camera frame passes through a YOLO detector and a pose model. The detector provides a direct posture or transition label. Pose keypoints provide a more structured representation of body motion. Tracking and stabilization reduce frame-to-frame changes, while the ST-GCN uses a short sequence rather than a single image. The final visual state is passed to the fusion engine together with confidence and timing information.

The repository includes two checkpoint files: a detector file of about 22.5 MB and a two-stream ST-GCN file of about 24.7 MB. The detector checkpoint has a published reference evaluation, reported in section 5; the temporal head uses upstream weights without deployment-domain fine-tuning, so its action accuracy is not characterized here. The dataset notes describe a primary source with 1,652 images: 1,286 for training, 266 for validation, and 100 for testing. Two private sources add 910 and 500 training images. Labels are remapped and perceptual duplicates are filtered.

There is an important limitation in this split. The available validation and test data are described as coming from the primary source, which has no held-out sitting examples. A four-class result would therefore be misleading unless a new test set covers all four classes. The merged image directory is also not present in the repository, which is why one detector smoke test is skipped.

3.3 Wearable path

The intended wristband combines an ESP32-S3, MAX30102, and IMU. In a finished system, it would send raw or processed PPG, signal quality, heart rate, and calibrated acceleration. The

Jetson receiver already supports message types for heart rate, acceleration, raw PPG, connection handshakes, and lost pulse contact.

The link is now specified as a written contract rather than an assumption. The wristband opens a TCP connection to the Jetson, which acts as the server on port 5555, and sends newline-delimited JSON. Each line is one typed event: an optional `hello` announcing device, firmware, and capabilities; `hr` carrying a pre-computed bpm; `accel` carrying `ax`, `ay`, and `az` in `g`; `ppg` carrying raw infrared and red sample bursts; and `pulse_lost` for loss of optical contact. Combined envelopes are explicitly rejected, because the receiver ignores unknown types. The checked-in firmware follows this contract and streams real accelerometer samples at 50 Hz; heart rate is still synthetic until the optical sensor is brought up, so `hr` values must not be read as physiological measurements.

Two capability flags in the Jetson configuration keep the system honest as hardware arrives. `has_accelerometer` controls whether the seizure-like branch can reach Tier 2 or degrades to Tier 1, and `has_ppg_raw` selects whether the Jetson derives beats per minute from raw bursts or trusts a pre-computed value. Because these flags gate escalation, they belong in the incident record alongside the tier.

One protocol limitation remains. The `ts` field is informational: the Jetson stamps each event with its own clock on receipt, so network delay cannot be separated from capture time. This matters because fusion is only meaningful when visual and wearable observations refer to the same event, and it is the reason we do not yet quote a cross-modal synchronization figure.

The wearable is therefore aligned at the transport and schema level and carries real motion data, but it is not yet fully integrated. MAX30102 bring-up is incomplete, so `hr` and `ppg` are synthetic rather than measured, and `has_ppg_raw` remains set accordingly. Packet loss, reconnect time, and battery endurance have not been characterised, and the firmware source still carries hard-coded Wi-Fi credentials that must be replaced with per-device provisioning before any use outside the laboratory.

3.4 Fusion and escalation

We chose explicit rules for the first prototype because they are easier to inspect than a learned fusion model. Each decision can record which channel contributed, how fresh it was, and why a tier was selected. The three tiers are summarized in [table 2](#).

Table 2 Prototype escalation tiers.

Tier	Interpretation	Typical evidence	Action
0	Observe or dismiss	Evidence does not satisfy an escalation rule. Missing data must not count as reassuring evidence.	Log the state and continue monitoring.
1	Verify	A credible possible fall without an immediate-severity combination, including camera-only operation.	Ask whether help is needed; escalate on help, silence, or continued hazard.
2	Escalate immediately	A configured combination labeled seizure-like or cardiac-risk together with a fall cue.	Skip the spoken delay and create an incident.

The names of Tier 2 branches are heuristic descriptions, not medical diagnoses. If acceleration is unavailable, the seizure-like test scenario falls back to Tier 1. A cardiac-risk heart-rate branch can still reach Tier 2. This is a useful example of graceful degradation, but it also means that a tier must always be displayed together with sensor availability.

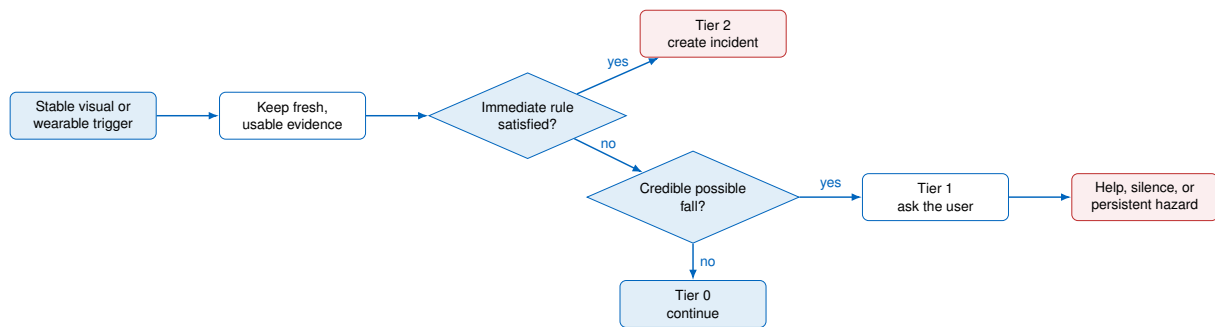


Figure 3 Fusion and verification flow. Every decision should retain evidence freshness and sensor availability.

3.5 Voice, WebRTC, and notification

Tier 1 gives the person a chance to say that they are safe or request help. The repository contains a real microphone/transcription path and a deterministic mock. The automated pipeline uses the mock, so it checks orchestration rather than speech recognition. A real evaluation should include room noise, distance, accents, unclear speech, timeouts, and microphone failure.

When an incident is created, the Jetson can publish video through `aiortc`, exchange WebRTC signaling over WebSockets, and expose session metadata to a Next.js dashboard and Expo mobile app. The current media path is video-only; audio is disabled in the example configuration. ICE configuration and TURN-oriented deployment templates exist, but cross-network success has not been demonstrated. Webhooks are also generic: tests mock the HTTP endpoint, and no emergency or messaging provider is currently integrated.

3.6 Web monitoring dashboard

The web dashboard is the caregiver’s primary view of an ongoing session. It is a Next.js application in which every component reads one normalized model, `MonitoringModel`, exposed under an `Overview`, `Vitals`, `Detection`, `Events`, and `Live sessions` navigation. The Jetson runtime publishes vitals as JSON at `GET /monitoring` on port 8790; a server-side adapter maps that envelope into the normalized model, and the browser polls it every three seconds. Keeping the model as the only contract means the transport can later become a WebSocket without rewriting interface components, and it keeps the dashboard independent of the wire format used by the edge pipeline.

Figure 4 shows the running overview. A status ribbon reports the edge node, wearable sensor, monitored location, and last synchronization time. A *current assessment* banner states the present interpretation (here, “All monitoring signals are steady”) together with the emergency and fall status, and changes tone with severity. The *vital & fall overview* panel shows heart rate in beats per minute and a fall-confidence ring; the *sensor fusion* panel lays out the wearable and vision inputs, the edge-fusion result, and the resulting decision; and the *response flow* panel tracks the fusion outcome, voice check, and alert dispatch. A filterable *recent events* timeline lists system, sensor, vision, fusion, voice, and alert entries, which are persisted in SQLite so that operational history survives a restart.

An earlier revision of the dashboard shipped with a staged mock adapter and a development-only scenario selector, which let the interface be designed before the edge pipeline could feed it. That path has since been removed: the dashboard now consumes the Jetson monitoring publisher only, so an unreachable edge node surfaces as an error state rather than as plausible-looking numbers. This is the safer behavior for a monitoring product, because it prevents a caregiver from reading synthetic vitals as real ones.

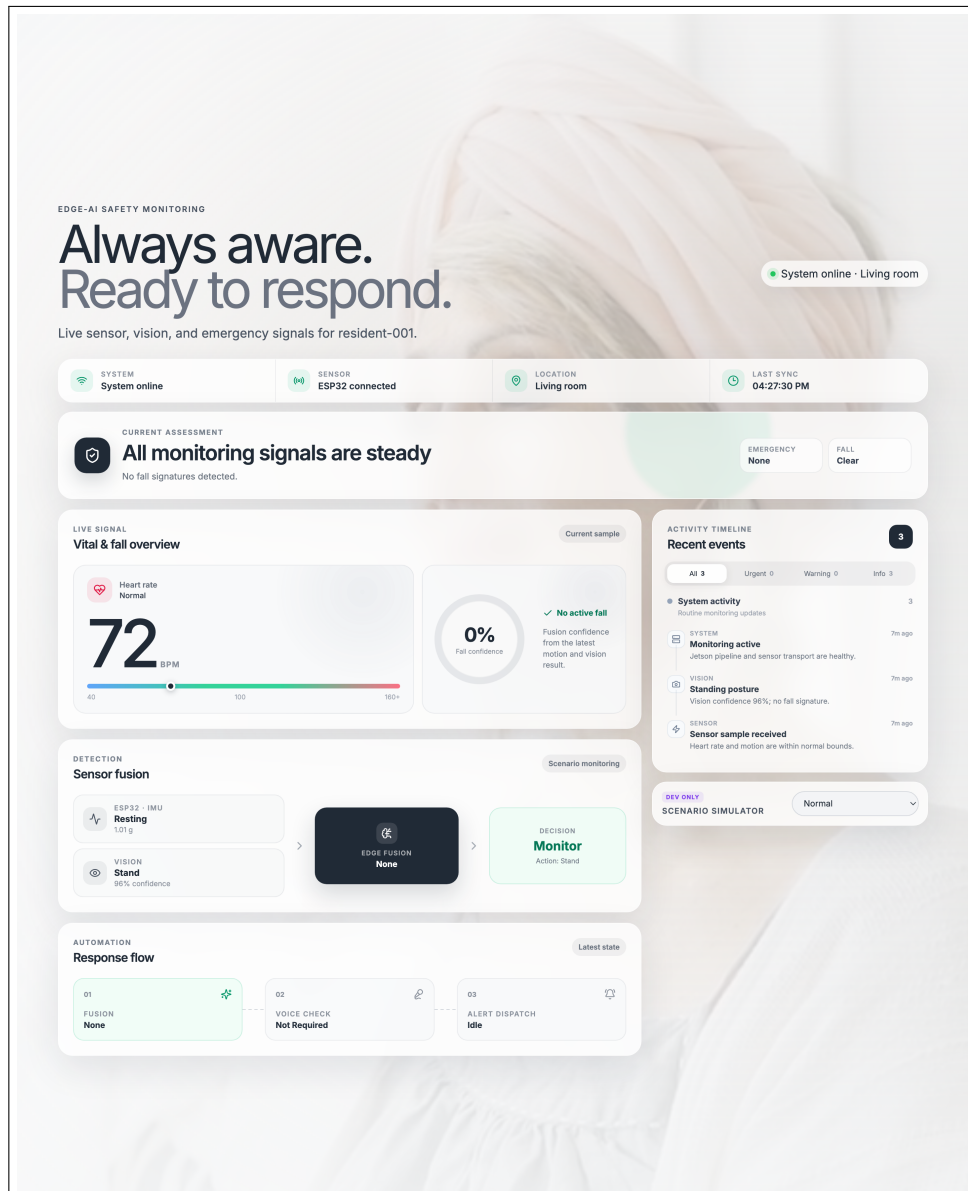


Figure 4 Web monitoring dashboard, overview view. Every panel reads the same normalized `MonitoringModel`, refreshed by polling every three seconds. This capture was taken during interface development, before the dashboard was repointed at a live Jetson.

Two boundaries in the dashboard are worth noting. Operational events are written through an `EventStore` interface with a SQLite implementation, so incident history survives a restart without introducing an external database. The `WebSocket` endpoint is deliberately left carrying WebRTC offer, answer, and ICE signaling only, not telemetry, which keeps the media path and the monitoring path independently debuggable.

4 Implementation

The edge software is organized as a set of small Python modules connected by an asynchronous pipeline. This made it possible for us to test individual decisions without a camera or wristband. It also keeps hardware-specific code at the edges of the system: camera capture produces frames, the sensor server produces typed updates, and downstream components work with normalized state.

Table 3 Implementation status at repository revision 07631fe.

Component	Status	Evidence and boundary
YOLO detection and pose	Evaluated	Validation mAP50 of 0.885 on three classes; no <code>sitting</code> test instances and no target-camera results.
Tracking and ST-GCN	Implemented in software	Feature, sequence, and stabilization logic are tested; upstream weights are used without deployment-domain fine-tuning.
Sensor TCP server	Implemented	Loopback framing is tested and the firmware now conforms to the documented NDJSON contract on port 5555.
Accelerometer	Integrated	Real samples stream at 50 Hz from the prototype board; the configuration flag must match the fitted hardware.
PPG processing	Prototype	Synthetic waveform tests pass; no real MAX30102 trace or reference-device comparison exists.
Fusion engine	Implemented in software	Representative normal, fall, seizure-like, cardiac-risk, and missing-sensor scenarios are tested.
Voice verification	Partial	Real and mock paths exist; automated integration uses the mock.
Monitoring dashboard	Implemented	Consumes the Jetson <code>/monitoring</code> endpoint on port 8790 with SQLite event history; no logs from a real incident.
WebRTC and clients	Partial	Server, signaling, dashboard, and mobile code exist; lab access relies on a Tailscale mesh and TURN remains unvalidated.
Webhook dispatch	Partial	Request and failure behavior are tested with mocked endpoints; no real provider is integrated.
Deployment	Prepared	<code>bootstrap.sh</code> and service templates exist; installation, recovery, and long-duration operation are unverified.

Camera capture supports USB, CSI/GStreamer, RTSP, and file inputs. This flexibility is useful for development, but each source has different latency and failure behavior. A final deployment should fix one camera path, document its geometry, and record dropped frames and reconnect time. The same principle applies to model files: every result should identify the exact checkpoint hash, configuration, test manifest, and repository revision.

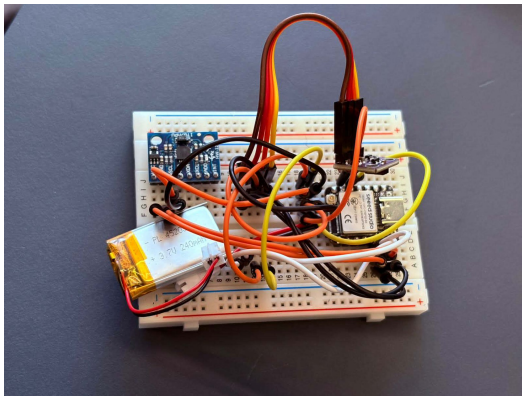
Security also needs attention. The audited ESP32 source contains a hard-coded Wi-Fi name, password, and server address. These values should be removed from source history and replaced with per-device provisioning. Remote clients should authenticate, signaling should be encrypted, tokens should be revocable, and incident video should be retained only for an explicit purpose. WebRTC protects media transport, but application-level authorization and data governance remain our responsibility [17].



(a) Assembled wristband enclosure with belt clip



(b) Opened enclosure with the prototype board inside



(c) Breadboard: XIAO ESP32-S3, MPU-6050 IMU, MAX30102, and a 3.7 V 240 mAh LiPo cell



(d) Jetson Orin Nano Developer Kit in a 3D-printed case with active cooling

Figure 5 Current hardware. The wearable is a breadboard prototype in a 3D-printed clip-on enclosure (a, b), carrying the ESP32-S3, IMU, optical sensor, and battery (c). The edge node (d) is a Jetson Orin Nano Developer Kit. The enclosure is a carried prototype rather than a wrist-worn strap, so sensor–skin contact for PPG is not yet in its final form.

5 Verification and Current Results

Two kinds of evidence are available at revision 07631fe: a reference evaluation of the detector checkpoint, and an automated test suite. The detector figures below are the repository's reference checkpoint metrics for `runs/detect/kp_v2_4cls/weights/best.pt` (YOLOv8s, 35 epochs, early stopping at epoch 15), reproducible with `scripts/eval.py`:

Validation mAP50 0.885 mAP50–95 0.556

We quote the validation split deliberately. The held-out test split reports a higher mAP50 of 0.977, but it is roughly 92% stand instances with only seven falling and one fallen example, so that number is inflated by class imbalance and should not lead. Per-class validation mAP50 is 0.900 for fallen, 0.835 for falling, and 0.921 for stand. Neither split contains sitting instances, so the fourth class is exercised only by live operation and the temporal head, and no four-class result can be claimed. falling remains the weakest class, which is expected for a transition rather than a posture.

The second strand of evidence is the automated test suite. It spans 17 test modules covering pose features, fusion rules, PPG processing, TCP transport, webhooks, monitoring payloads, voice overrides, temporal stabilisation, and end-to-end pipeline smoke tests; the repository's report pack records 89 collected tests at this revision. We describe the suite's composition rather than quoting a pass count, because it was not re-executed on the machine used to prepare this document.

Both strands are useful, but they need careful interpretation. The tests establish that many software components behave consistently on controlled inputs. Pose features, temporal utilities, post-processing, and stabilization are checked. Fusion tests cover representative scenarios and missing acceleration. PPG tests use generated waveforms. TCP tests use real loopback sockets. Webhook tests use mocked responses. Pipeline smoke tests use no physical camera, a mock speech recognizer, synthetic sensors, and one in-process TCP wristband emulator.

The skipped detector smoke test needs the absent merged test-image directory. Beyond the detector metrics quoted above, the suite does not establish real heart-rate quality, Jetson frame rate, alert latency, microphone performance, remote WebRTC success, notification delivery, or battery endurance. A validation score and a passing mocked pipeline are evidence of component readiness, not evidence that a real fall will be detected in a deployment room.

Table 4 What the current verification does and does not show.

Area	Established	Still missing
Vision logic	Feature shapes, post-processing, stabilization, and validation mAP50 of 0.885 on three classes.	A sitting-bearing test manifest, confusion matrix, event-level recall, deployment-camera failures.
Fusion	Expected tier for constructed scenarios and for a missing accelerometer.	Field-calibrated thresholds, false alarms, missed falls, synchronized real inputs.
Wearable	Contract-conformant firmware, real 50 Hz accelerometer stream, parser framing, synthetic PPG behavior.	Measured PPG from the optical sensor, capture timestamps, Wi-Fi loss, signal quality, endurance.
Caregiver view	Dashboard reading the live Jetson monitoring endpoint; SQLite event history.	Screenshots and logs from a real incident on a deployed edge node.
Interaction	Mock voice and webhook control flow.	Real speech, provider delivery, recipient acknowledgement, failure recovery.
Remote viewing	WebRTC and client implementation; Tailscale-based lab access.	Authenticated cross-network STUN/TURN trials and reconnect results.

5.1 Evaluation plan

The final model evaluation should use a frozen test manifest with all four detector classes and difficult negatives such as rapid sitting, intentional lying, bending, partial occlusion, and poor lighting. It should report per-class precision, recall, F1 score, support, and a confusion matrix. Because `falling` is a transition, event-level recall and time from physical onset to stable detection are more informative than frame accuracy alone. The detector, pose/ST-GCN path, and fused system should also be compared as separate ablations.

End-to-end trials should record physical event onset, visual decision, sensor arrival, fusion decision, prompt start, interpreted response, incident creation, first remote frame, webhook request, and confirmed receipt. Results should be distributions, including a 95th percentile, rather than a single successful demonstration. Fault trials should disconnect the camera, wristband, network, client, and remote endpoint. Missing data must be shown as unavailable or stale, never as a normal measurement.

6 Limitations and Future Work

The present limitation is straightforward: KineticPulse is more complete as an integrated pipeline than as a measured system. The wearable now speaks the documented contract and streams real acceleration, but heart rate is still synthetic until the optical sensor is brought up, and the device is a clip-on breadboard prototype rather than a wrist-worn strap, so PPG contact conditions are not representative. Because the Jetson stamps events on receipt rather than at capture, we also cannot yet quantify how well camera and wearable evidence are aligned in time.

The visual models need a reproducible evaluation set. The available split does not support a fair four-class result, and no repository-contained confusion matrix or deployment-video benchmark exists. Likewise, the real microphone path, external WebRTC path, and webhook recipient path remain untested. The project therefore cannot yet support claims about accuracy, latency, reliability, privacy, or emergency response.

Our next steps are ordered by dependency:

1. finish real MAX30102 acquisition so that heart rate is measured rather than synthetic;
2. add device time, sequence numbers, units, freshness, signal quality, and reconnect behavior;
3. freeze a representative model test set and publish machine-readable predictions and metrics;
4. instrument the Jetson pipeline for latency, FPS, dropped frames, resource use, and thermals;
5. test speech in representative rooms and define clear help, safe, unknown, timeout, and failure states;
6. deploy authenticated signaling with STUN/TURN and test web and mobile clients on separate networks;
7. integrate a named notification provider or keep the feature explicitly described as a generic webhook;
8. run a controlled scenario and fault matrix, followed by battery and long-duration tests; and
9. complete credential provisioning, a threat model, retention rules, consent, and an audit trail.

For staged falls, participant safety is more important than realism. Trials should use healthy informed participants, protective equipment, supervision, and an approved protocol. If representative testing is not possible, the final claims should stay limited to the laboratory setup.

7 Conclusion

KineticPulse connects several useful ideas into one understandable workflow: edge vision notices a possible fall, a wearable adds motion and, in time, physiological context, explicit rules choose an escalation tier, voice gives the person a chance to respond, and caregiver clients receive an incident session. Over the semester the project moved from a set of separately working modules to a connected path: the wearable now speaks a documented TCP contract and streams real acceleration, the detector has a published validation score, and the caregiver dashboard reads the Jetson's live monitoring endpoint instead of staged data.

The remaining work is equally clear. Heart rate must be measured rather than synthesized; the detector needs a frozen test manifest that includes the fourth class; and the complete path must be timed on the Jetson with real speech, remote clients, and notification delivery. Until then, the honest result is an integrated laboratory prototype, not a validated fall detector or emergency system. That distinction makes the project stronger: it leaves a precise, testable route from working code to credible evidence.

References

- [1] World Health Organization, “Falls.” Fact sheet, Apr. 2021. Accessed: 2026-08-01.
- [2] M. Mubashir, L. Shao, and L. Seed, “A survey on fall detection: Principles and approaches,” *Neurocomputing*, vol. 100, pp. 144–152, 2013.
- [3] N. T. Newaz and E. Hanada, “The methods of fall detection: A literature review,” *Sensors*, vol. 23, no. 11, p. 5212, 2023.
- [4] A. Ramachandran and A. Karuppiah, “A survey on recent advances in wearable fall detection systems,” *BioMed Research International*, vol. 2020, p. 2167160, 2020.
- [5] E. Alam, A. Sufian, P. Dutta, and M. Leo, “Vision-based human fall detection systems using deep learning: A review,” *Computers in Biology and Medicine*, vol. 146, p. 105626, 2022.
- [6] C. Taramasco, C. Pineiro, D. Ormeño Arriagada, L. Robles, and D. Araya, “Multimodal dataset for sensor fusion in fall detection,” *PeerJ*, vol. 13, p. e19004, 2025.
- [7] P. Qi, D. Chiaro, and F. Piccialli, “FL-FD: Federated learning-based fall detection with multi-modal data fusion,” *Information Fusion*, vol. 99, p. 101890, 2023.
- [8] Ultralytics, “Yolov8 documentation,” 2026. Accessed: 2026-08-01.
- [9] S. Yan, Y. Xiong, and D. Lin, “Spatial temporal graph convolutional networks for skeleton-based action recognition,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [10] Espressif Systems, “ESP32-S3 series datasheet,” 2026. Accessed: 2026-08-01.
- [11] Analog Devices, “MAX30102 high-sensitivity pulse oximeter and heart-rate sensor for wearable health,” 2026. Accessed: 2026-08-01.
- [12] P. H. Charlton *et al.*, “The 2023 wearable photoplethysmography roadmap,” *Physiological Measurement*, vol. 44, no. 11, p. 111001, 2023.
- [13] M. Satyanarayanan, P. Bahl, R. Caceres, and N. Davies, “The case for VM-based cloudlets in mobile computing,” *IEEE Pervasive Computing*, vol. 8, no. 4, pp. 14–23, 2009.
- [14] NVIDIA Corporation, “Jetson orin nano developer kit user guide,” 2026. Accessed: 2026-08-01.
- [15] SYSTRAN, “faster-whisper: Faster whisper transcription with ctranslate2.” Software repository, 2026. Accessed: 2026-08-01.
- [16] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” *arXiv preprint arXiv:2212.04356*, 2022.
- [17] World Wide Web Consortium, “WebRTC: Real-time communication in browsers,” recommendation, W3C, Mar. 2025.
- [18] A. Keranen, C. Holmberg, and J. Rosenberg, “Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal,” Tech. Rep. RFC 8445, Internet Engineering Task Force, July 2018.